

From API Requirements to OpenAPI Reference Documentation

A docs-as-code API documentation project using OpenAPI, JSON, Git, and Redocly

PROJECT TYPE

API documentation / docs-as-code

DELIVERABLES

OpenAPI specification · JSON examples · rendered API reference

TOOLS

GitHub · OpenAPI · Redocly CLI · Node.js / npm · JSON

Scott Harmer

Technical Writer

Portfolio 2026

PROJECT OVERVIEW

This portfolio project demonstrates how I transformed a set of API requirements into structured, developer-facing reference documentation using a docs-as-code workflow. I used OpenAPI, JSON, Git, and Redocly to define, validate, and render a fictional Task Management API.

My objective was to demonstrate the complete documentation process, from interpreting source requirements and designing the API structure to producing machine-readable specifications, reusable schemas, JSON examples, and a rendered API reference.

THE CHALLENGE

The challenge was to turn a set of functional API requirements into documentation that was both machine-readable and useful to developers. The source requirements described what the Task Management API needed to support, but they did not yet provide the structure, schemas, examples, status codes, or reusable components required for a complete API reference.

I therefore needed to interpret the requirements, define a clear API structure, document each endpoint consistently, and ensure that the resulting OpenAPI specification could be validated and rendered through a docs-as-code workflow.

SOURCE REQUIREMENTS

I began with a short set of functional requirements for a fictional Task Management API. These requirements described the actions the API needed to support, but they did not yet define the endpoint structure, request and response schemas, authentication requirements, error handling, or reusable data models.

My first task was to turn those requirements into a clear documentation structure that could support both human readers and machine processing through OpenAPI.

REQUIREMENT	API NEED
Create a task	POST /tasks
Retrieve all tasks	GET /tasks
Retrieve one task	GET /tasks/{taskId}
Update a task	PATCH /tasks/{taskId}
Delete a task	DELETE /tasks/{taskId}
Filter tasks by status or project	Query parameters on GET /tasks

This set of requirements provided the basis for the API design, endpoint documentation, schemas, examples, and validation work that followed.

MY ROLE

For this portfolio project, I acted as the technical writer responsible for transforming the source requirements into a complete API documentation structure.

My work included interpreting the functional requirements, defining the endpoint structure, documenting request and response behaviour, creating reusable schemas and JSON examples, and organising the content into an OpenAPI specification that could be validated and rendered as developer-facing reference documentation.

DOCS-AS-CODE APPROACH

I structured the API documentation as a docs-as-code project so that the source content could be written, versioned, validated, and rendered using the same workflow commonly used for software documentation.

Rather than maintaining the API reference as a static document, I organised the content as plain-text source files using OpenAPI, YAML, JSON, and Markdown within a Git-based repository.

WORKFLOW



This approach made the documentation maintainable, traceable, and suitable for automated validation and publishing.

API DESIGN & JSON

I translated the functional requirements into a small REST API with clearly defined resources, methods, parameters, and response behaviours. I also used JSON examples to show developers the structure of the data returned by the API.

The design focused on consistency across endpoints, reusable data models, predictable status codes, and examples that could be understood quickly alongside the OpenAPI specification.

ENDPOINT DESIGN

METHOD	ENDPOINT	PURPOSE
POST	/tasks	Create a new task
GET	/tasks	Retrieve and filter tasks
GET	/tasks/{taskId}	Retrieve one task
PATCH	/tasks/{taskId}	Update an existing task
DELETE	/tasks/{taskId}	Delete a task

JSON RESPONSE EXAMPLE

```
{
  "id": "task_1024",
  "title": "Review API documentation",
  "status": "in_progress",
  "priority": "high",
  "projectId": "project_210"
}
```

Using structured endpoint definitions and consistent JSON examples helped connect the source requirements to a developer-facing API model.

VALIDATION & QUALITY ASSURANCE

I validated the OpenAPI specification to identify structural errors, inconsistencies, and documentation issues before rendering the final API reference.

Using Redocly linting helped me check that the specification followed the expected OpenAPI structure and that reusable components, paths, parameters, and responses were defined consistently.

QUALITY CHECKS

OPENAPI STRUCTURE

Paths, methods, components

SCHEMA CONSISTENCY

Reusable data models

RESPONSE COVERAGE

Success and error responses

LINTING

Redocly validation

Validation provided an additional quality-control layer and helped ensure that the source documentation could be rendered reliably as a developer-facing API reference.

FINAL API REFERENCE

The completed OpenAPI specification was designed to render as structured developer-facing reference documentation. Each operation brings together the endpoint purpose, HTTP method, path parameters, request and response information, status codes, and JSON examples in a consistent format.

REFERENCE EXCERPT

Search...

POST

Create a task

GET

Retrieve tasks

GET

Retrieve a task

PATCH

Update a task

DEL

Delete a task

Task Management API (1.0.0)

Download OpenAPI specification: [Download](#)

License: LicenseRef-Proprietary

A fictional REST API for creating, retrieving, updating, filtering, and deleting tasks.

Create a task

Creates a new task in the task management system.

AUTHORIZATIONS: > [bearerAuth](#)

REQUEST BODY SCHEMA: [application/json](#)
required

title

required

string

status

string

priority

string

Responses

> 201 Task created successfully.

< 400 Invalid request.

POST

/tasks

Request samples

Payload

Content type
application/json

Copy

```
{
  "title": "Review API documentation",
  "status": "in_progress",
  "priority": "high"
}
```

Response samples

201

Content type
application/json

Rendered Redocly API reference generated from the validated OpenAPI specification.

The rendered reference demonstrates how the OpenAPI source can support a navigable developer-facing interface while remaining part of a version-controlled docs-as-code workflow. The same specification was also validated with Redocly CLI and bundled into a machine-readable JSON file.

RESULTS / IMPACT

The completed project transformed a short set of functional requirements into a structured API documentation set covering five operations. The final OpenAPI specification passed Redocly validation with zero errors and zero warnings, and was used to generate both a machine-readable JSON bundle and a rendered HTML API reference.

TOOLS & SKILLS

TOOLS & TECHNOLOGIES	SKILLS DEMONSTRATED
OpenAPI 3.1	API documentation
YAML and JSON	Structured content authoring
GitHub	Version control and change tracking
Redocly CLI	OpenAPI validation and rendering
Node.js / npm	Documentation tooling
macOS Terminal	Command-line workflow
	Requirements analysis
	REST API structure and endpoint design
	Schema and response documentation
	Docs-as-code workflow
	Documentation quality assurance

View the full project on GitHub:

<https://github.com/Scott-Harmer/task-management-api>